

```
In [1]: %matplotlib inline
```

决策树实验

实验目的

掌握决策树学习算法。

实验要求

编程实现基于基尼指数进行划分选择的未剪枝决策树学习算法。

上机内容1

结合搜索引擎阅读并理解以下代码。

并回答：以下代码和课上所授决策树内容有什么区别？

注意，请将以下代码中random_state替换为自己学号。

```
In [2]: %reset -f
# 导入所需的库
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score

# 加载鸢尾花数据集
iris = load_iris()
X = iris.data # 特征矩阵
y = iris.target # 类别向量
feature_names = iris.feature_names # 特征名称

# 划分训练集和测试集
random_state = 12345
np.random.seed(random_state) # 设置随机种子，保证可复现性
indices = np.random.permutation(len(X)) # 生成随机索引
split = int(len(X) * 0.8) # 设置划分比例为80%
X_train = X[indices[:split]] # 训练集特征
y_train = y[indices[:split]] # 训练集类别
X_test = X[indices[split:]] # 测试集特征
y_test = y[indices[split:]] # 测试集类别

# 定义计算基尼指数的函数
def gini(y):
    """
    输入：类别向量y
    输出：基尼指数
    """
    # 注释？
```

```

unique, counts = np.unique(y, return_counts=True)
freqs = counts / len(y)
# 注释?
gini = 1 - np.sum(freqs**2)
return gini

# 定义根据特征和阈值划分数数据集的函数
def split_dataset(X, y, feature_index, threshold):
    """
    输入: 特征矩阵X, 类别向量y, 特征索引feature_index, 阈值threshold
    输出: 划分后的左右子集(X_left, y_left), (X_right, y_right)
    """
    # 根据特征和阈值对数据进行二分划分
    left_indices = X[:, feature_index] <= threshold # 左子集的索引
    right_indices = X[:, feature_index] > threshold # 右子集的索引
    X_left = X[left_indices] # 左子集特征
    y_left = y[left_indices] # 左子集类别
    X_right = X[right_indices] # 右子集特征
    y_right = y[right_indices] # 右子集类别
    return (X_left, y_left), (X_right, y_right)

# 定义寻找最佳划分特征和阈值的函数
def best_split(X, y):
    """
    输入: 特征矩阵X, 类别向量y
    输出: 最佳划分特征best_feature, 最佳划分阈值best_threshold, 最佳划分基尼指数best_gini
    """
    # 初始化最佳划分参数
    best_feature = None
    best_threshold = None
    best_gini = 1 # 最大可能的基尼指数为1

    n_features = X.shape[1] # 特征的数量

    for feature_index in range(n_features): # 遍历每个特征
        feature_values = X[:, feature_index] # 获取该特征的所有取值
        possible_thresholds = np.unique(feature_values) # 获取该特征的所有可能的阈值

        for threshold in possible_thresholds: # 遍历每个阈值
            # 根据该特征和阈值划分数数据集为左右两个子集
            (X_left, y_left), (X_right, y_right) = split_dataset(X, y, feature_index,
                                                                    threshold)
            if len(y_left) == 0 or len(y_right) == 0: # 如果某个子集为空, 则跳过该划分
                continue
            # 计算左右子集的权重和基尼指数
            weight_left = len(y_left) / len(y)
            weight_right = len(y_right) / len(y)
            gini_left = gini(y_left)
            gini_right = gini(y_right)
            # 计算该划分的加权基尼指数
            weighted_gini = weight_left * gini_left + weight_right * gini_right
            # 如果该划分的基尼指数小于当前最佳划分的基尼指数, 则更新最佳划分参数
            if weighted_gini < best_gini:
                best_feature = feature_index
                best_threshold = threshold
                best_gini = weighted_gini

    return best_feature, best_threshold, best_gini

# 定义构建决策树的函数

```

```

def build_tree(X, y, max_depth=5, min_samples_split=2):
    """
    输入：特征矩阵X，类别向量y，最大深度max_depth，最小划分样本数min_samples_split
    输出：决策树，以字典的形式表示
    """

    # 创建一个空字典，用于存储决策树的信息
    tree = {}

    # 如果节点中的数据属于同一类别，则将节点标记为叶子节点，并返回其类别标签
    if len(np.unique(y)) == 1:
        tree["type"] = "leaf"
        tree["class"] = y[0]
        return tree

    # 如果节点的深度达到了最大深度，则将节点标记为叶子节点，并返回其数据中出现最多的类别标签
    if max_depth == 0:
        tree["type"] = "leaf"
        tree["class"] = np.bincount(y).argmax()
        return tree

    # 如果节点的数据量小于最小划分样本数，则将节点标记为叶子节点，并返回其数据中出现最多的类别标签
    if len(y) < min_samples_split:
        tree["type"] = "leaf"
        tree["class"] = np.bincount(y).argmax()
        return tree

    # 否则，寻找最佳划分特征和阈值，并将节点分为左右两个子节点
    best_feature, best_threshold, best_gini = best_split(X, y)

    # 如果没有找到合适的划分，则将节点标记为叶子节点，并返回其数据中出现最多的类别标签
    if best_feature is None or best_threshold is None:
        tree["type"] = "leaf"
        tree["class"] = np.bincount(y).argmax()
        return tree

    # 否则，根据最佳划分特征和阈值划分数据集为左右两个子集
    (X_left, y_left), (X_right, y_right) = split_dataset(X, y, best_feature, best_threshold)

    # 将节点标记为内部节点，并存储其划分特征和阈值
    tree["type"] = "internal"
    tree["feature"] = feature_names[best_feature]
    tree["threshold"] = best_threshold

    # 递归地对左右子节点进行同样的操作，减少最大深度
    tree["left"] = build_tree(X_left, y_left, max_depth-1, min_samples_split)
    tree["right"] = build_tree(X_right, y_right, max_depth-1, min_samples_split)

    return tree

# 定义根据决策树对新数据进行预测的函数
def predict(tree, x):
    """
    输入：决策树tree，单个样本x（一维数组）
    输出：预测结果y_pred（整数）
    """

    # 如果当前节点是叶子节点，则返回其类别标签作为预测结果
    if tree["type"] == "leaf":
        return tree["class"]

```

```

# 否则, 根据当前节点的划分特征和阈值将样本分配到左右子节点中
feature_index = feature_names.index(tree["feature"]) # 获取划分特征的索引
threshold = tree["threshold"] # 获取划分阈值

if x[feature_index] <= threshold: # 如果样本的特征值小于等于阈值, 则分配到左子节点
    return predict(tree["left"], x)
else: # 否则, 分配到右子节点
    return predict(tree["right"], x)

# 使用训练数据集构建决策树
tree = build_tree(X_train, y_train, max_depth=3, min_samples_split=5)

# 使用测试数据集进行预测, 并计算准确率
y_pred = [predict(tree, x) for x in X_test]
accuracy = accuracy_score(y_test, y_pred)
print("The accuracy of the decision tree is:", accuracy)

```

The accuracy of the decision tree is: 0.8666666666666667

上机内容2

基于以下代码框架和上述示例代码实现DecisionTreeClassifier（做适当修改），将其用于西瓜数据集2.0的分类，并计算准确率。

```

In [3]: %reset -f
class DecisionTreeClassifier(object):
    def __init__():
        # 初始化
        pass

    def fit(X, y):
        # 训练决策树
        pass

    def predict(X):
        # 预测决策树
        pass

```